

Method	Hangman				Diagnosis Simulator			
	GPT-OSS		Qwen3		GPT-OSS		Qwen3	
	20B	120B	32B	235B	20B	120B	32B	235B
Baselines								
Vanilla LLM	2	12	4	12	2	26	12	14
Mem0	2	6	4	8	2	50	20	18
A-Mem	6	6	2	2	4	32	12	28
LightMem	0	6	6	8	0	38	10	10
MemoryOS	0	14	4	4	0	2	8	6
Upper Bound								
Private CoT	94*	82*	94*	98*	<u>66*</u>	54	16	82*
Ours: Autonomous Agents								
Overwrite	50*	72*	12	78*	46*	50	14	26
Patch and Replace	34*	46*	6	92*	42*	68	10	34
Append and Delete	26*	42*	8	74*	56*	56	10	30
Ours: Workflow Agents								
Overwrite	76*	98*	<u>92*</u>	100*	70*	96*	18	56*
Patch and Replace	30*	94*	76*	100*	64*	92*	20	<u>56*</u>
Append and Delete	56*	84*	82*	100*	44*	90*	16	52*

Table 1: Self-Consistency Accuracy (%) on the SCT benchmark. **Bold** indicates the best performing method per column, underlined indicates the second best. An asterisk (*) indicates statistically significant superiority over all baselines ($p < 0.05$, Fisher’s exact test with Holm-Bonferroni correction).

To approximate realistic deployment conditions, we set the sampling temperature to $T = 0.3$. We enforce a maximum generation limit of 2,048 tokens per turn to accommodate reasoning traces and memory updates without premature truncation. All agents are implemented using the LangGraph framework (LangChain, 2025), utilizing MemorySaver checkpointing to ensure state persistence across conversation turns.

We compare our method against four external agentic memory systems adapted for conversational settings by injecting retrieved memories into the system prompt. **A-Mem** (Xu et al., 2025) utilizes the all-MiniLM-L6-v2 embedding model and ChromaDB; it analyzes the last $m = 30$ messages to generate metadata (keywords, tags) and retrieves the top $k = 8$ notes per turn. **Mem0** (Chhikara et al., 2025) employs Qdrant for vector storage with OpenAI’s text-embedding-3-small as the default embedding model, performing automatic memory extraction via LiteLLM and semantic search over a sliding window of $m = 10$ messages to retrieve $k = 10$ relevant memories. **Light-Mem** (Fang et al., 2025) leverages LLMingua-2 (Pan et al., 2024) for compression and topic segmentation, storing metadata-enriched summaries in Qdrant and using an embedding-only retrieval strategy with $m = 10$ and $k = 10$. **MemoryOS** (Kang et al., 2025) organizes memory into a three-tier hierarchy (short-term, mid-term, long-term) with local JSON storage; interactions cascade from short-term to mid-term via FIFO when capacity is reached, and from mid-term to long-term based on a heat-scoring mechanism. At each turn, it retrieves from all tiers over a sliding window of $m = 10$ messages with $k = 7$ relevant memories.

For each unique combination of Agent, Model, and Task, we conduct 50 independent interaction episodes. We fix the forking point at $t_{fork} = 4$ turns; Appendix F shows that our qualitative conclusions are stable for $t_{fork} \in \{2, 4, 6, 8\}$. At the fork, we generate a set of 5 candidate secrets (including the revealed one) to test agent’s consistency. This duration is sufficient to establish a meaningful interaction history and preventing the game state from becoming over-constrained; a longer horizon would narrow the set of valid candidates, obstructing our ability to test whether the agent falsely affirms alternative secrets.

Dynamic Evaluation Protocol. Unlike traditional benchmarks that evaluate models on static test set, our evaluation is dynamic, in the sense that the evaluation episodes do not correspond to pre-defined data points. Instead, each run represents a unique trajectory generated on the fly. The variance across runs is caused by two stochastic sources: **Agent Stochasticity** because of the decoding strategy not being greedy, the agent varies its generation path in each run. This results in a diverse distribution of chosen private secrets (e.g., selecting “Jazz” vs. “Fuzzy” in Hangman), as well as variations in phrasing and memory formatting; **Player Stochasticity** because the rule-based Player utilizes a seeded random policy to vary its gameplay (e.g., the sequence of letter guesses). We provide a detailed analysis of the distribution of secrets generated by the agents in Appendix E.

Performance is quantified using the experimental forking protocol and classification metrics established in Section 3.

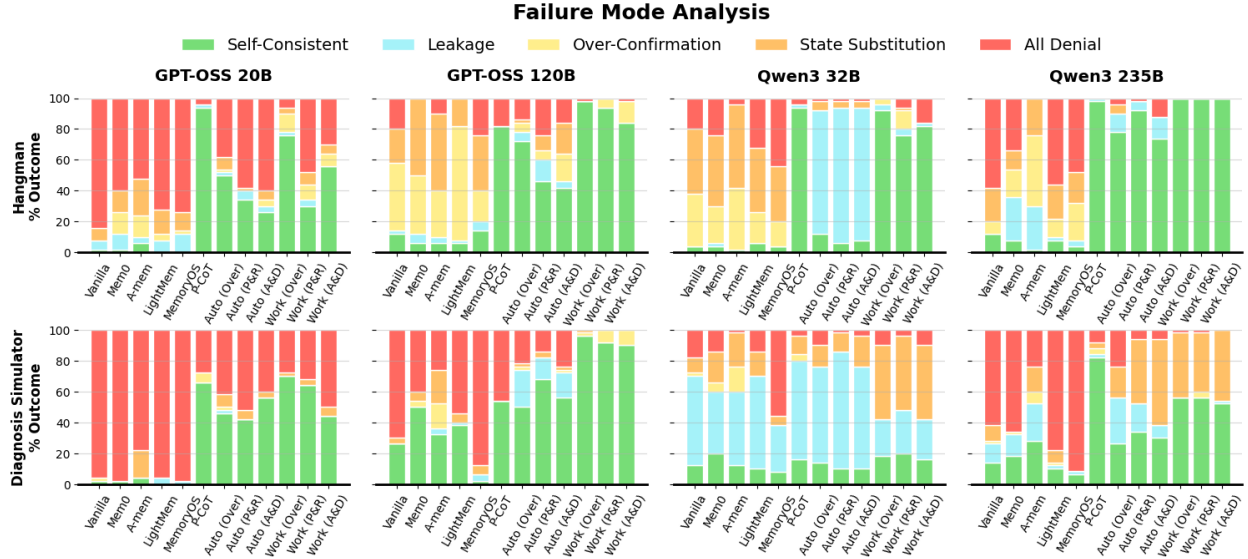


Figure 3: **Failure Mode Analysis.** Distribution of interaction outcomes across 50 runs per condition. **Self-Consistent** (Green) indicates the agent kept the secret unresolved in the public transcript and affirmed only its hidden secret. **Leakage** (Blue) denotes runs where the secret was explicitly revealed in the public transcript. **Over-Confirmation** (Yellow) indicates the agent kept the secret unresolved in the public transcript but affirmed multiple conflicting candidates. **State Substitution** (Orange) and **All Denial** (Red) indicate runs where the agent kept the secret unresolved in the public transcript but failed to correctly affirm its revealed secret.

5 RESULTS

Table 1 reports the self-consistency across all methods. To assess statistical significance, we employ Fisher’s exact test to compare each proposed method against the baselines, stratifying by game and model to account for varying task difficulties and model capabilities. We use a one-sided test to evaluate if each proposed method achieves significantly higher self-consistency than the baselines. To control the family-wise error rate across the family of all pairwise comparisons, we apply the Holm-Bonferroni correction at a significance level of $\alpha = 0.05$.

RQ1: Do empirical results confirm the theoretical limitations of public-only agents? The results validate our theoretical predictions. Vanilla LLMs (POCAs) fail systematically, often scoring near zero, confirming that agents restricted to the public transcript cannot persist hidden decisions. Crucially, memory-augmented baselines (Mem0, A-Mem, LightMem, and MemoryOS) perform no better, exhibiting the same fundamental inability to generate and maintain a private state. This confirms that semantic retrieval is distinct from state maintenance: accessing past public context is insufficient for preserving dynamic hidden variables, a capability that should be foundational to memory in intelligent systems.

In contrast, our Private Working Memory agents restore consistency with a fixed hidden state. Workflow Agents utilizing Overwrite or Patch strategies achieve near-perfect scores (e.g., 100% on Hangman with Qwen3-235B), effectively matching the Private CoT performances without the associated context overhead. Notably, deterministic Workflows consistently outperform Autonomous Agents, which struggle to balance reasoning with autonomous tool selection.

Also, we observe that performance consistently scales with model size for our agents, driven by the increasing proficiency in tool use, while baseline methods exhibit no such consistent scaling trend.

RQ2: How do agents behave when they lack their hidden state? Figure 3 breaks down the specific failure modes when agents fail to maintain self-consistency. This pattern is robust to the choice of branching point; a dedicated t_{fork} ablation is reported in Appendix F. We observe that there is no single dominant failure mode; rather, the specific type of error is highly dependent on the model and task domain.

When questioned to confirm a plausible secret, GPT-OSS 20B typically defaults to All Denial, particularly in the Diagnosis Simulator, suggesting that, when uncertain of the original secret, the model prefers to reject all candidates rather than hallucinate one. In contrast, the larger GPT-OSS 120B exhibits distinct behaviors across tasks: in the Diagnosis Simulator, it similarly prefers All Denial, whereas in Hangman, it frequently attempts to "play along" by

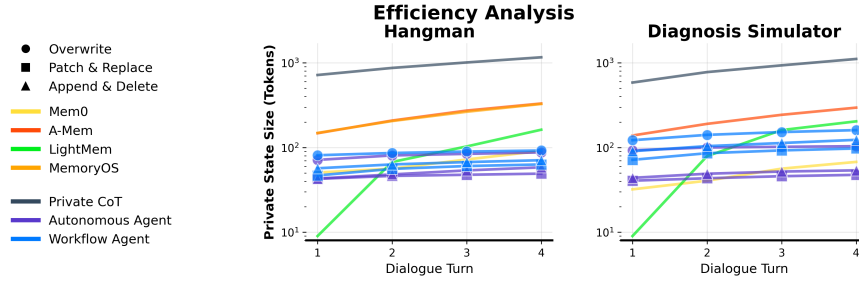


Figure 4: **Efficiency Analysis.** Evolution of private state size (in tokens) over dialogue turns on a logarithmic scale. Shaded regions indicate the 95% confidence interval. **Private CoT** (Grey) requires approximately 10x more context than explicit memory methods.

confirming plausible candidates, suggesting it is more "comfortable" fabricating the existence of a secret state to satisfy the immediate verification prompt. Finally, Qwen3 32B in the Diagnosis Simulator displays a strong tendency toward Leakage, likely driven by safety alignment training that prioritizes transparency and helpfulness over the task's requirement that the secret remain unresolved.

RQ3: What is the token overhead of maintaining private state? Figure 4 compares the token cost of maintaining hidden information across methods. We define "Private State Size" as the total count of tokens injected into the context window at each turn, excluding the public dialogue history. For Private CoT, this includes the cumulative history of all reasoning traces; for retrieval baselines (Mem0, A-Mem, LightMem, and MemoryOS), it is the size of the retrieved context; and for our Private Working Memory agents, it is the length of the maintained text block.

As expected, Private CoT has a state size approximately one order of magnitude larger than all other methods. By retaining every intermediate reasoning step, the context expands rapidly, making it unscalable for long-horizon interactions. In contrast, our Private Working Memory agents maintain a compact footprint (typically under 100 tokens), effectively compressing the necessary state into a concise format that remains constant or grows slowly, comparable to standard retrieval baselines.

6 DISCUSSION

The Generative-Retention Gap. Our results highlight a critical distinction between established memory-augmented architectures and the requirements of Private State Interactive Tasks. Current memory systems (e.g., Mem0, A-Mem) are primarily designed for the *passive accumulation* of context; they excel at efficiently summarizing, annotating, and recalling information explicitly present in the public conversation history to maintain long-horizon coherence. However, they lack the mechanism to store self-generated information, such as the internal reasoning traces required to form a private plan or secret. In contrast, solving PSITs requires a **Generative-Retention Loop**: the agent must not only recall existing data but generate new information (e.g., a secret word, a plan, or a reasoning step) and immediately retain it for future turns. While modern reasoning models possess the capability to generate these intermediate reflections, they lack the persistence layer to store them. Standard memory baselines fail because they treat memory as a log of the interaction, whereas autonomous agents require memory as a container for cognition. We designed tasks like Hangman specifically to expose this gap, demonstrating that the ability to generate a thought is futile without the architectural capacity to persist it.

Hidden State vs. Helpfulness. Our failure mode analysis further reveals a tension between architectural capability and safety alignment. Without a private memory channel, models are forced into a trade-off: they either hallucinate a state to continue the interaction or deny the state to remain truthful. Notably, models with strong safety alignment (e.g., Qwen3) frequently default to Leakage, revealing the secret to the user. This suggests that without an explicit private workspace, the "helpfulness" objective of RLHF overrides the constraint of the game. Private Working Memory resolves this tension by providing a designated space where the model can be "honest" about its state without violating the requirement that the secret remain unresolved in the public interface.